

Bitcoin Index Time Series Analysis

Minh Phuc Le

April 30, 2025

Contents

1	Introduction	3
1.1	Background and Significance	3
1.2	Research Objectives	3
1.3	Data and Computational Tools	3
1.4	Analytical Approach	3
2	Preliminary Exploration	4
2.1	Data Summary	4
2.2	Basic Statistics	4
2.3	Time Series Plot	5
2.4	Decomposition	6
2.5	Scatter Plot	7
2.6	Summary of Descriptive Analysis	7
3	Stationarity Check	8
3.1	Stationarity Formal Tests	9
3.2	Autocorrelation Analysis	10
3.3	Summary of Stationarity Check	10
3.4	Normality Test	11
4	Model Specification	12
4.1	Data Transformation	12
4.2	Order of Differencing	15
4.3	Model Identification using ACF and PACF	17
4.4	Extended ACF (EACF) Analysis	18
4.5	BIC Table	18

5	Fit selected models	19
5.1	Coefficient significance	19
5.2	Model Selection Based on Information Criteria	20
5.3	Forecast Accuracy Evaluation	21
5.4	Summary of Model Fitting	21
6	Conclusion	22
6.1	Key Findings	22
6.2	Limitations	22
6.3	Future Research Directions	22
7	Appendix: R Code	23

1 Introduction

1.1 Background and Significance

The quick advance of financial markets has raised Bitcoin and other cryptocurrencies to prominence in both scholarly and practical research domains. Bitcoin demonstrates extensive price volatility together with dynamic trends since its launch which makes it a suitable choice for time series analysis. The analysis of Bitcoin price changes remains vital because it permits reliable financial predictions and protective risk measures and valuable strategic investments. Time series models specifically ARIMA models serve as a structured method to extract data structures and create precise predictions through historical record analysis. The study helps explain Bitcoin time-dependent characteristics while duplicating practical time series modeling techniques that financial analysts utilize in their work.

1.2 Research Objectives

This report uses ARIMA modelling to analyze Bitcoin index data (USD) from August 2011 to January 2025 that occurs monthly. The main goals of this research consist of:

- A detailed descriptive analysis evaluates the Bitcoin price series.
- Multiple suitable ARIMA(p,d,q) models will be identified through established model specification methods
- Model fitting and goodness-of-fit criteria evaluation decides the selection of these models
- An assessment will determine the best model for both interpretation and forecasting through the combination of statistical and visual diagnostics.

1.3 Data and Computational Tools

The analyzed dataset contains Bitcoin index data from US dollars which spans from August 2011 to January 2025 in monthly intervals. The analysis uses an extensive 12-year period which captures multiple market phases that include growing phases and phases with consolidate. The study makes use of R programming language for analysis while utilizing its statistical and graphical resources. RStudio functions as the integrated development environment to maintain code reproducibility while ensuring both clarity and efficiency of programming.

1.4 Analytical Approach

The research starts by performing an exploratory data analysis to present main features of the series through visual displays and statistical summaries. The model specification process utilizes four diagnostic tools which include the autocorrelation function (ACF) and partial autocorrelation function (PACF) together with extended autocorrelation function (EACF) and Bayesian Information Criterion (BIC) for proposing candidate ARIMA models. The proposed models receive data input for parameter estimation which leads to analysis of findings within the underlying process framework. Models are evaluated through evaluation metrics AIC, BIC and MSE until an optimal selection for forecasting purposes is achieved. A complete documentation in the Appendix contains all R scripts and functions while preserving transparency and reproducibility.

2 Preliminary Exploration

2.1 Data Summary

```
##      Date Bitcoin
## 1 08/2011 10000.0
## 2 09/2011  6025.0
## 3 10/2011  4125.0
## 4 11/2011  3987.5
## 5 12/2011  5725.0
## 6 01/2012  6625.0

## [1] "data.frame"

## Time-Series [1:162] from 2012 to 2025: 10000 6025 4125 3988 5725 ...

## [1] 0
```

The R program executed successfully to read “assignment2Data2025.csv” into a data.frame containing Date and Bitcoin columns.

By executing ts() function, the time series object named data.ts was generated while establishing August 2011 as the initial period and monthly frequency accounting for twelve observations each year. The str() output shows the data.ts object has 162 observations which cover the period from August 2011 through early 2025.

The function sum(is.na(data.ts)) revealed no missing values in the data.ts dataset since it returned a value of zero. No imputation or data cleaning processes need to happen before performing time series modeling because the data is complete in its current state.

2.2 Basic Statistics

Table 1: Descriptive Statistics of Bitcoin Index

	mean	sd	min	max	skew	kurtosis
X1	19781743	27959247	3987.5	128015000	1.66	2.25

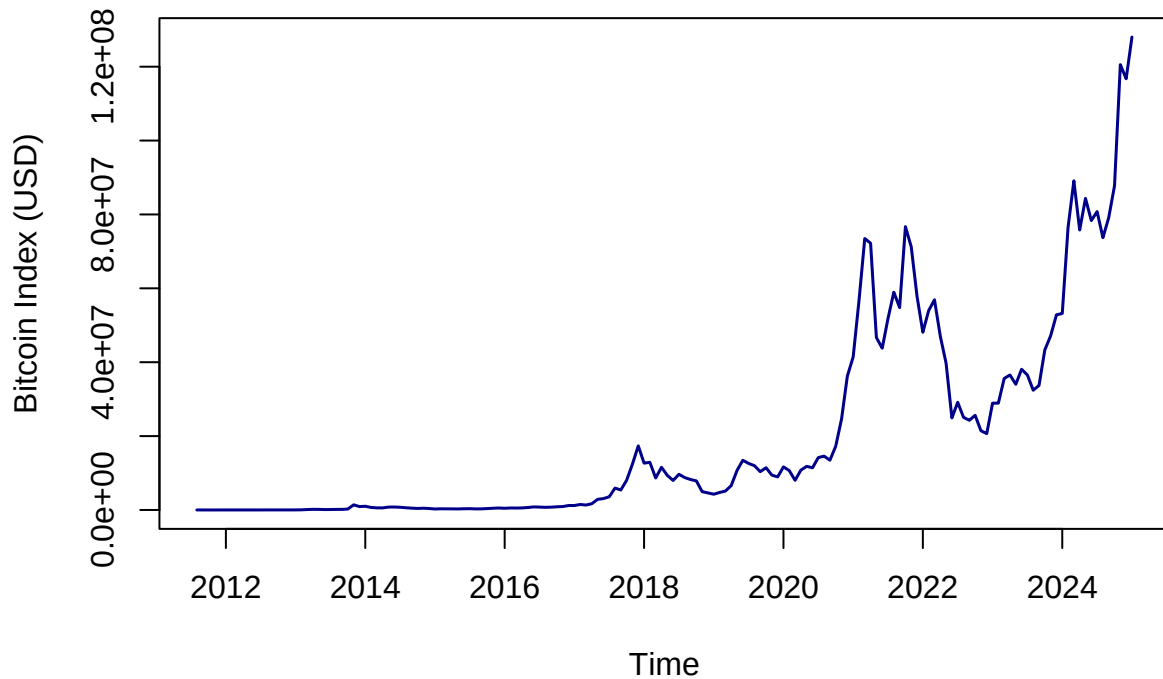
Table 1 demonstrates the main statistical metrics which characterize Bitcoin Index values during August 2011 through early 2025. Bitcoin Index values exhibit significant time-based variability as their mean rests at 19.78 million and the standard deviation measures 27.96 million.

Historical Bitcoin performance displayed a wide spectrum of variation between 3,988 and 128 million with the minimum and maximum recorded values respectively.

A skewness value of 1.66 demonstrates a distribution of Bitcoin prices with positive skewness which means longer right tails containing more extensive large values. The distribution is mildly platykurtic because its value of 2.25 shows flatness compared to normal distributions without significant deviation from normal distribution characteristics regarding “tailedness.”

2.3 Time Series Plot

Figure 1. Time Series Plot of the Bitcoin Index



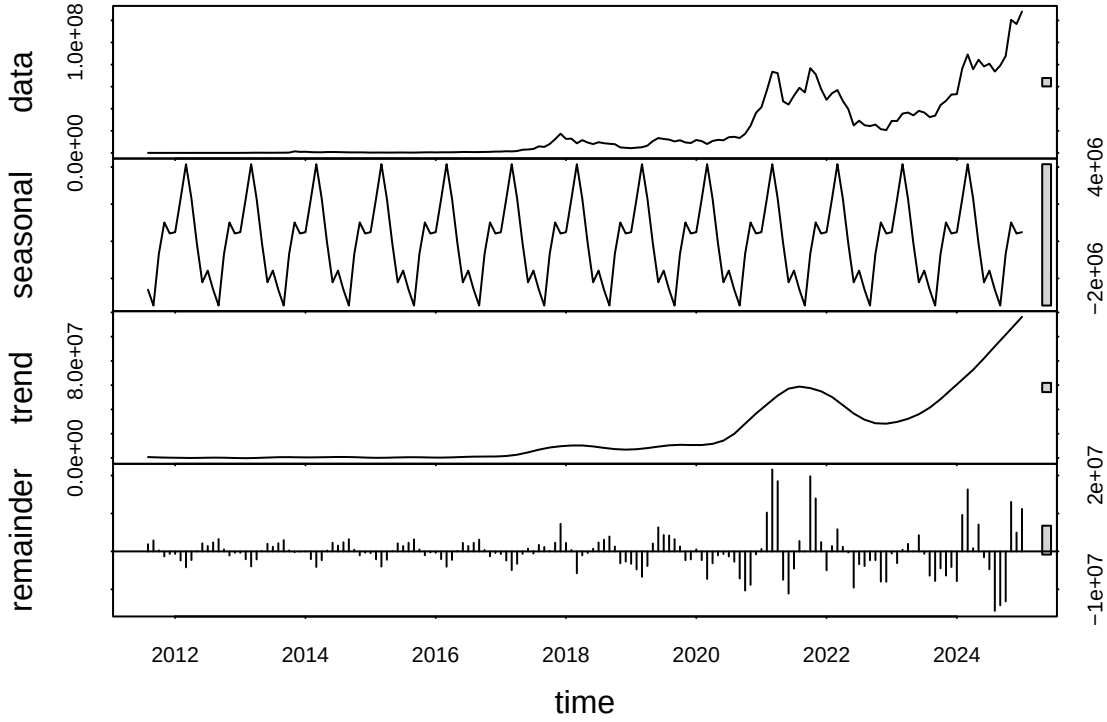
The Bitcoin Index chart spans from August 2011 up until early 2025 as depicted in Figure 1. The series shows ongoing upward movement most prominently during the 2017 period forward. The Bitcoin value has been increasing steadily throughout the study period but maintained significant fluctuation points within this pattern.

A consistent annual pattern is not highlighted in the initial data examination period. The time series shows dominant volatility clusters that emerge first during 2017-2018 and then reappear starting in 2020 when it produces successive large price fluctuations. Market activity and uncertainty levels tend to reach peaks at such times.

Multiple structural breaks emerge in the data series starting from 2017 and continuing through 2021 which produced significant dramatic shifts in time series behavior. The cryptocurrency market underwent key developments which produced equivalent signals in the Bitcoin Index along with shifts in Bitcoin's adoption methods during this period.

2.4 Decomposition

Figure 2. STL Decomposition of Bitcoin Index



The Bitcoin Index receives STL (Seasonal-Trend-Loess) decomposition analysis in Figure 2. The decomposition technique divides a time series into three distinct sections which include the trend element and seasonal effects and remaining variables.

The trend component analysis validates the upward price movement evident in the original Bitcoin Index series which elevated rapidly in 2017-2018 then surged again between 2020 and 2022. Over different time spans the trend grows at varying speeds before and after joining the market.

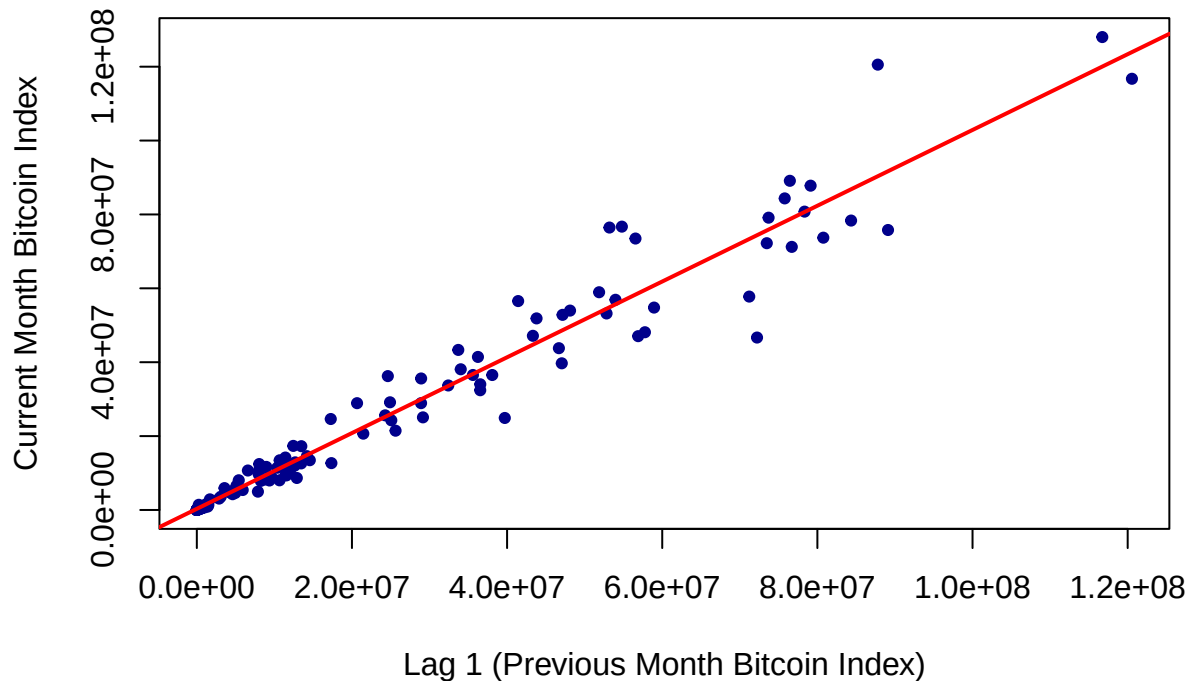
The seasonal pattern produces limited changes which remain small when compared to the size of the trend. Seasonal price patterns of Bitcoin remain insignificant when analyzing the entire price trajectory along with random fluctuations. The seasonality remains steady and uniform over the studied period.

The residual element has substantial variability because it produces larger deviations when Bitcoin prices increase. A transformation would benefit the analysis because the residuals exhibit heteroscedasticity which means their variances are inconsistent across the data points.

The trend component dominates the Bitcoin Index together with irregular fluctuations above seasonality and its effects.

2.5 Scatter Plot

Figure 3. Scatter Plot of Neighboring Pairs (Lag 1)



`## Correlation between current and previous month Bitcoin values: 0.9779`

The connection between Bitcoin Index and its one-month time lag appears in Figure 3. The observed strong positive correlation of 0.9779 reveals that the time series displays high persistence because present values closely depend on previous month values. The strong autocorrelation at lag 1 makes this series exhibit signs of non-stationarity.

Most of the plotted points maintain a tight alignment with the regression line especially during lower value ranges of the data. The relationship becomes less stable as Bitcoin prices increase because there is greater variance in the statistical data points. The observations demonstrate heteroscedasticity because of their changing dispersion pattern from earlier patterns.

The procedure produces a sizable correlation coefficient of 0.9779 indicating that consecutive data points exhibit practically perfect association characterizing patterns with prominent trends. The substantial autocorrelation value at lag 1 requires differencing in order to proceed with ARIMA modeling.

2.6 Summary of Descriptive Analysis

Various essential properties stand out in the Bitcoin Index time series after conducting a detailed descriptive analysis.

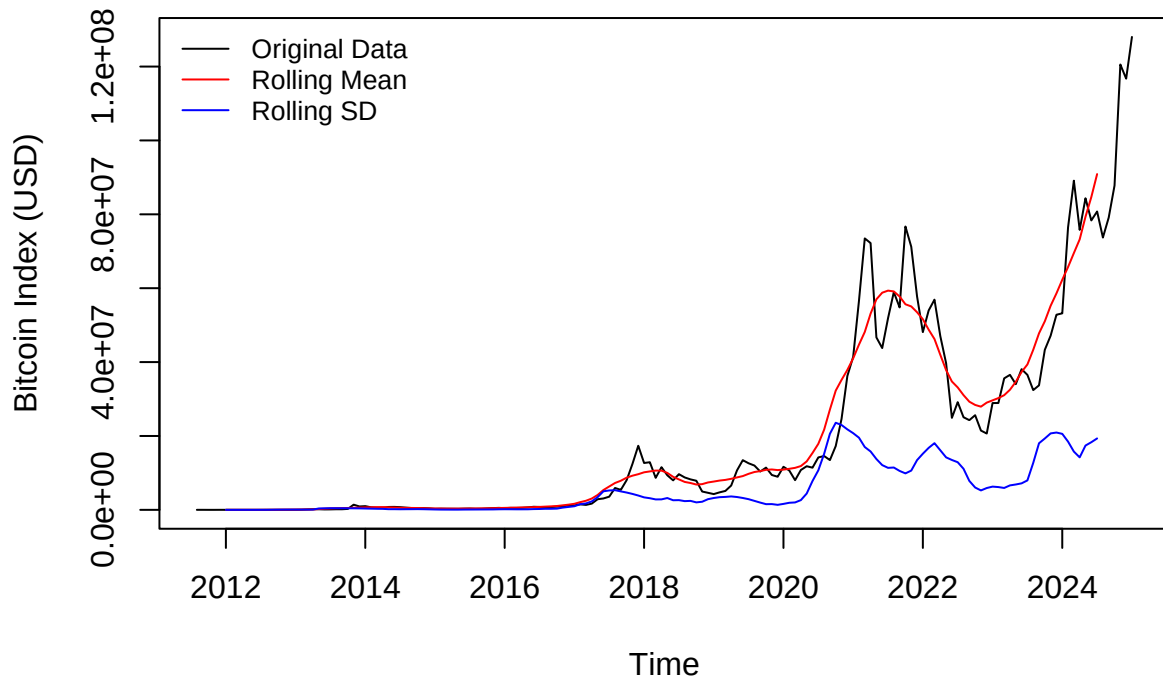
1. **Strong Upward Trend:** The data shows clear upward movement following 2017 which manifests as both raw and STL decomposition confirms.

2. **Changing Variance:** The data demonstrates distinct heteroscedasticity because the series volatility raises directly in proportion to its values. A variances stabilizing transformation should probably be implemented for this data.
3. **Minimal Seasonality:** The extracted seasonality factor by STL decomposition operates at a very small scale in relation to the trend and irregular components thus indicating weak seasonality in this particular financial data set.
4. **High Autocorrelation:** Extremely high correlations between consecutive observations (0.9954) reveal that this series demonstrates strong time-dependent characteristics of non-stationary data.
5. **Potential Structural Breaks:** Several noticeable regime changes emerge in the time series starting from 2017 and continuing through 2020-2021 that might reflect critical events within the cryptocurrency market.
6. **Non-Normal Distribution:** The positive skewness value of 1.66 shows a non-normal pattern in which the right end extends further than the left end thus requiring transformational methods for normality distribution.

Before modeling with ARIMA the Bitcoin Index requires transformation and differencing because both techniques are required to transform its non-stationary tendencies. This part will systematize the required analysis strategy.

3 Stationarity Check

Figure 4. Rolling Statistics of Bitcoin Index



The time series of Bitcoin Index appears in black on Figure 4 while the red line represents the 12-month rolling mean and the blue line shows rolling standard deviation.

The Bitcoin Index shows significant growth in an upward direction after 2017 because it shows rising volatility across its changing values. The rolling mean adjusts to the underlying data patterns which signifies there is no stationarity in the data mean.

The rolling standard deviation indicates substantial variability throughout the evaluation period where volatility rises sharply beginning from 2020. The existence of heteroskedasticity demonstrates non-constant variance thus representing a sign of non-stationarity.

Before fitting an ARIMA model to the Bitcoin Index data one must first transform the series because it exhibits nonstationarity through nonconstant mean and variance over time.

3.1 Stationarity Formal Tests

Table 2: Results of Stationarity Tests

	Test	Statistic	p_value	Null_Hypothesis	Conclusion
Dickey-Fuller	Augmented Dickey-Fuller	-0.2705646	0.9900000	Non-stationary	Fail to Reject
KPSS Level	KPSS	2.3395273	0.0100000	Stationary	Reject
Dickey-Fuller Z(alpha)	Phillips-Perron	-2.4274780	0.9557147	Non-stationary	Fail to Reject

The results from Table 2 demonstrate convincing evidence that the Bitcoin index data series is not stationary.

1. ADF Test Interpretation

- A time series contains a unit root when the ADF test maintains its null hypothesis indicating non-stationarity.
- The computed test statistic of -0.27056 along with a p-value of 0.99 leads to non-rejection of the null hypothesis.
- The time series probably demonstrates nonstationarity since it contains potential trends or varying time-dependent variance measurements.

2. KPSS Test Interpretation:

- Per the KPSS test null hypothesis the time series exists as stationary data either around mean or around a trend.
- The results show a test statistic value of 2.3395 that produces a p-value under 0.01 therefore leading to rejection of the null hypothesis of stationarity.
- The obtained test results serve to prove that the time series contains non-stationary elements.

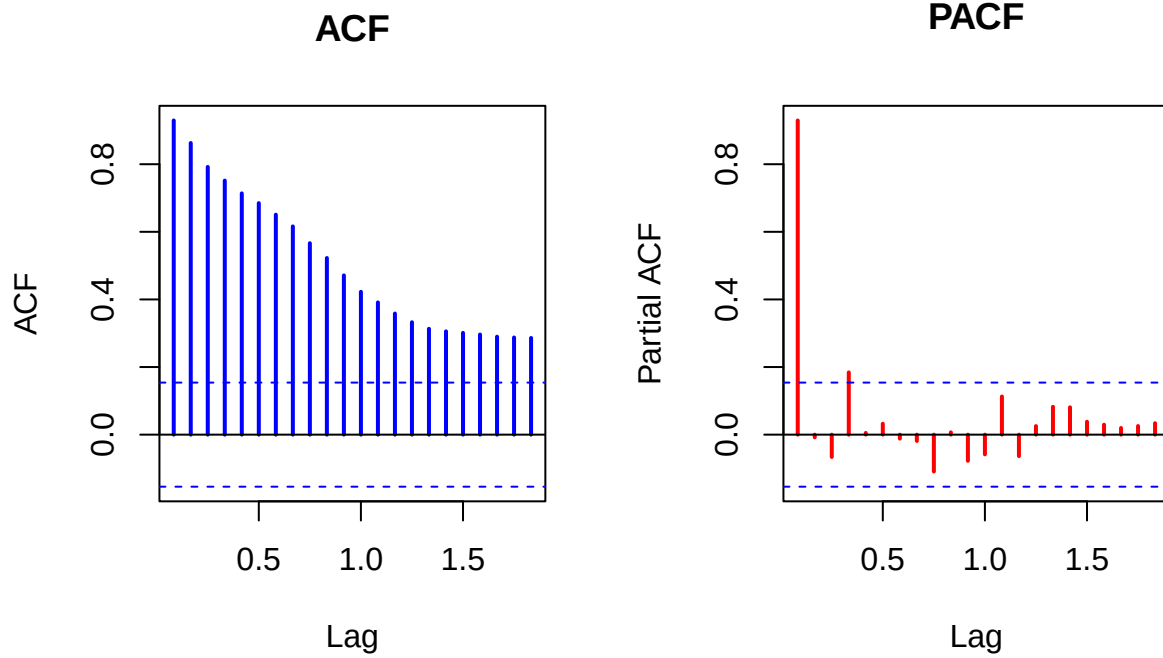
3. PP Test Interpretation

- The null hypothesis in the PP test defines the time series to be non-stationary because it contains a unit root.
- The test statistic value of -2.4275 along with the p-value of 0.9557 indicates very high results.
- The results from the p-value emerge as higher than 0.05 which leads to the sustainment of the null hypothesis thus indicating that the series remains non-stationary.

Each of the utilized tests prove that Bitcoin index data exists in a non-stationary state. A transformation of data by differencing is needed to create stationary data which enable proper ARIMA modeling.

3.2 Autocorrelation Analysis

Figure 5 – Bitcoin Index ACF & PACF



The Bitcoin Index series displays its autocorrelation patterns through Figure 5 which contains both ACF and PACF plots. Autocorrelation exists strongly throughout many lag terms in the ACF results because the decay pattern manifests as a slow gradual decrease. The long-lasting decline while approaching zero indicates that the data represents a non-stationary time series pattern. The first lag of the PACF plot produces a significant value followed by negligible values at higher lags indicating that autocorrelation predominantly happens at this first order.

Both ACF and PACF plots showcase substantial initial autocorrelation findings which indicate the Bitcoin Index data is non-stationary before going through transformation. The finding follows the initial analysis of rolling mean/variance plots that were conducted before.

First-order differencing ($d = 1$) appears necessary according to these graphs to achieve stability in the series. A proper AR and MA specification for ARIMA modeling should be done by examining both ACF and PACF after differencing.

3.3 Summary of Stationarity Check

The Bitcoin Index time series underwent a complete stationarity investigation with both graphical analysis and formal statistical procedures.

The rolling mean and standard deviation plots indicated that the data was non-stationary because both the mean demonstrated an upward pattern and the variance showed rising value especially following 2017. An upward trend together with increasing volatility in the series confirms the existence of both trend non-stationarity and heteroskedasticity without constant mean and variance.

The statistical results demonstrated that the data is not stationary.

- The results from the Augmented Dickey-Fuller (ADF) test indicated a unit root remains present because the p-value reached 0.99 and rejected stationarity.
- The results from the KPSS test validated the presence of non-stationarity since it rejected the null hypothesis (p-value = 0.01).
- Additional evidence pointing to non-stationarity of the series emerged from the Phillips-Perron (PP) test as the null hypothesis of a unit root could not be rejected (p-value = 0.9557).

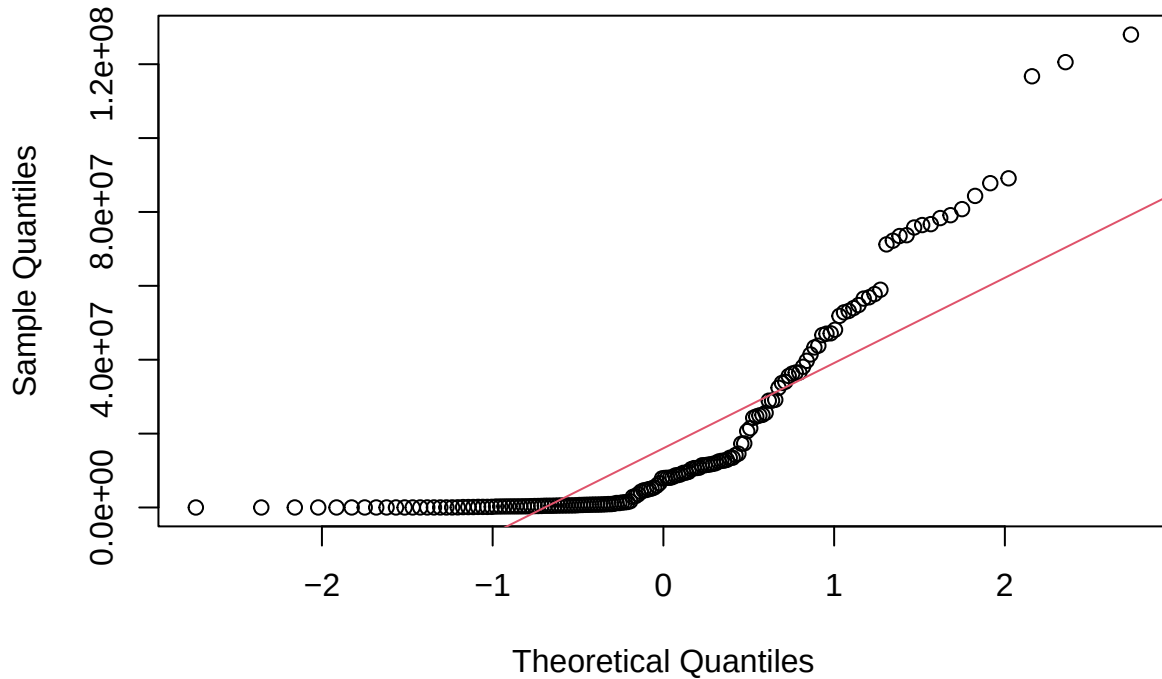
These findings received confirmation through the analysis of autocorrelation (ACF) and partial autocorrelation (PACF) plots. Analysis of the ACF plot revealed a slow descending pattern that indicates non-stationarity whereas the PACF displayed a strong lag 1 spike together with quick decay patterns.

The Bitcoin Index series shows no evidence of stationarity according to all examined results. The first step to perform ARIMA modeling requires first-order differencing ($d = 1$) since it allows achieving stationarity. A stabilizing transformation should be performed before differencing because heteroskedasticity exists in the data. Additional analysis of AR and MA terms for modeling will depend on reevaluating ACF and PACF traces after performing both transformations and differencing steps.

3.4 Normality Test

A normality assessment of Bitcoin index time series involved executing a Q-Q plot visualization together with the Shapiro-Wilk normality test.

Figure 6. Normal Q-Q Plot of the Original Time Series



Shapiro-Wilk normality test: W = 0.7379 , p-value = 1.102e-15

Q-Q Plot Analysis

Figure-6 compares the quantiles of the dataset against those of a theoretical normal distribution. A dataset that follows the normal distribution pattern would create data points that stay close to the 45-degree reference axis. The extreme deviation points at heavy-tailed distributions together with skewness patterns due to the observed data points. This visual evidence confirms that normality does not apply to the examined dataset.

Shapiro-Wilk Test Results

Since the p-value occurs below 0.05 we dismiss the null hypothesis concerning normality distribution. The assessment confirms that Bitcoin index data deviates from following a normal distribution.

Implications for Modeling

Before conducting ARIMA modeling it becomes necessary to transform the data since its non-normal distribution indicates unstable variance.

4 Model Specification

ARMA and ARIMA models represent standard approaches for modeling temporal data while forecasting series containing time-dependent observations. Both mean and variance elements of the Bitcoin Index demonstrate evidence of non-stationarity throughout the data series. The application of ARMA models is restricted because it needs the data to be stationary. BY using differencing in the ARIMA model the data transforms from non-stationary to stationary states which makes forecasting and analysis possible.

Therefore, ARIMA is the most appropriate modeling framework for the Bitcoin Index dataset.

4.1 Data Transformation

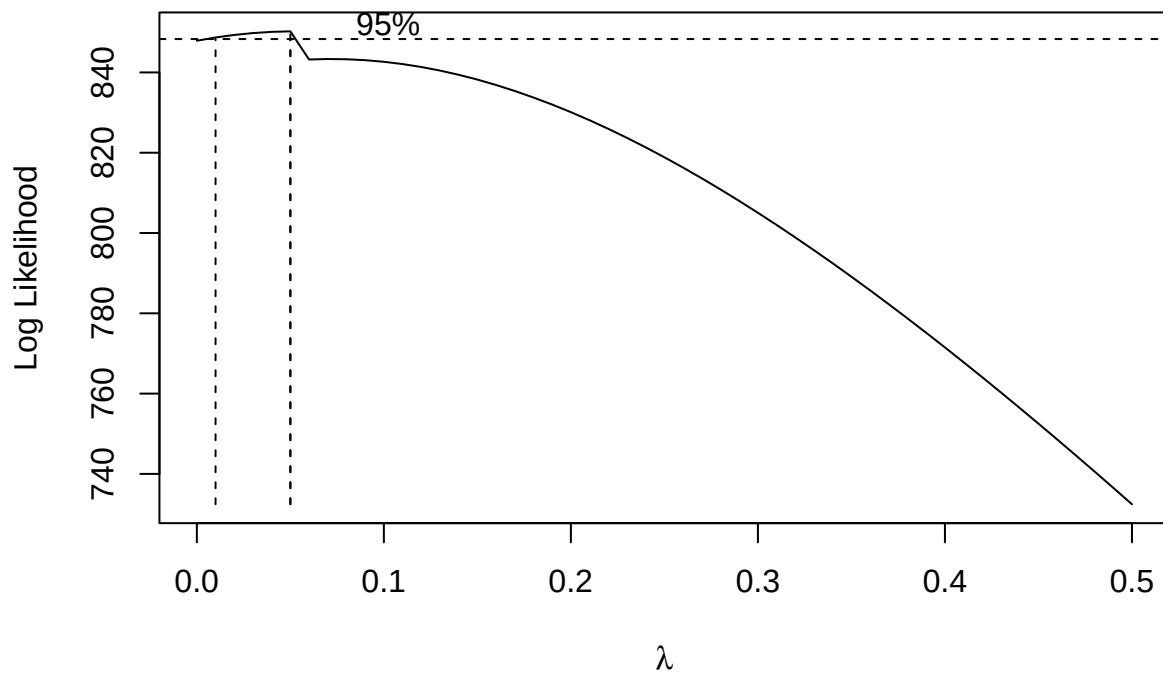
Figure 1 shows that Bitcoin displays strong overall upward movement while its variance increases progressively thereby proving that both mean and variance are non-stationary. Stationarity tests together with rolling statistics confirm that the examined series exists in a non-stationary state.

Before applying differencing to stabilize the mean, it is essential to first stabilize the **variance**, which appears to grow proportionally with the level of the series. This is a characteristic often addressed through **Box-Cox transformation**.

A check for positive values must precede Box-Cox transformation because the algorithm cannot process negative values or zeroes. Assessing the dataset for negative entries is essential for Box-Cox transformation because it cannot handle negative or zero values.

```
## [1] 3987.5
```

Dataset suitability for Box-Cox transformation testing remains valid because the positive minimum values show no need for specific adjustments. As part of attempts to find suitable limitations the Box-Cox transformation showed 0 and 0.5 functioned as proper power transformation constraints.

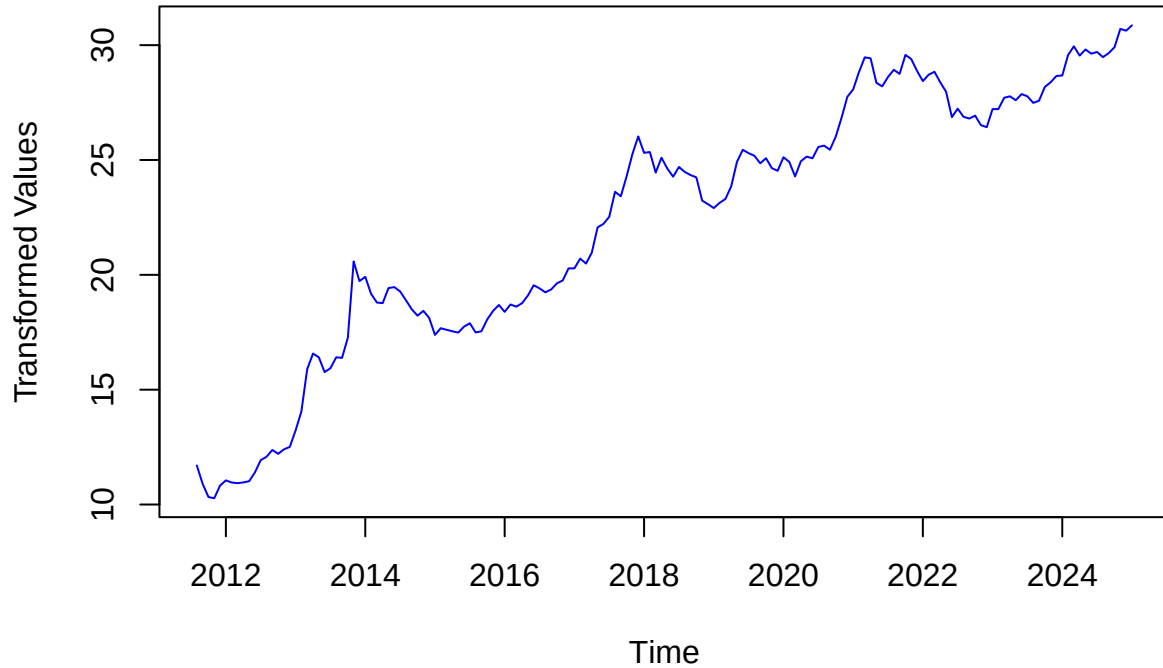


Based on the Box-Cox estimation the optimal lambda displayed high precision through its narrow 95% confidence interval. The peak of the log-likelihood curve has irregular shape but the optimization warnings do not affect the validity of the statistical estimate provided by the confidence interval. The Box-Cox transformation will now be implemented with this optimal lambda value.

```
## [1] 0.01 0.05
```

```
## [1] 0.05
```

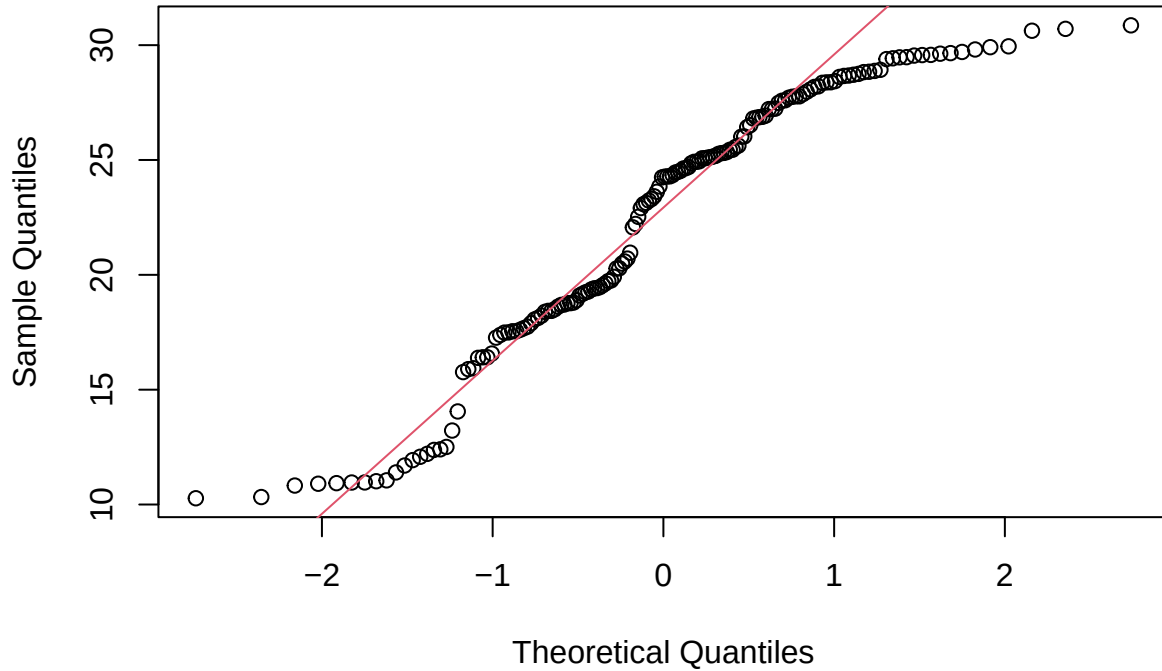
Figure 7. Box-Cox Transformed Time Series



The original Bitcoin Index series (Figure 1) displays exponential expansion together with rising variance levels which implies both non-stationarity and heteroskedasticity. ARIMA modeling becomes unreliable because these traits break down its fundamental modeling assumptions.

The analysis implemented a Box-Cox transformation using lambda value near 0.05 to solve this issue. The outcome series shows a straighter pattern and maintains consistent volatility levels in Figure 7 which implies better adherence to ARIMA requirements. Necessary first-step variance stabilization through transformation makes it possible to develop better predictive models which produce more trustworthy results. The present analysis employs the transformed series for fitting an ARIMA model due to better performance.

Figure 8. Normal Q–Q Plot for the Box–Cox transformed series



```
## Shapiro-Wilk normality test after Box-Cox: W = 0.9325 , p-value = 6.461e-07
```

```
##
```

```
## ADF test after Box-Cox: Statistic = -2.4479 , p-value = 0.3895
```

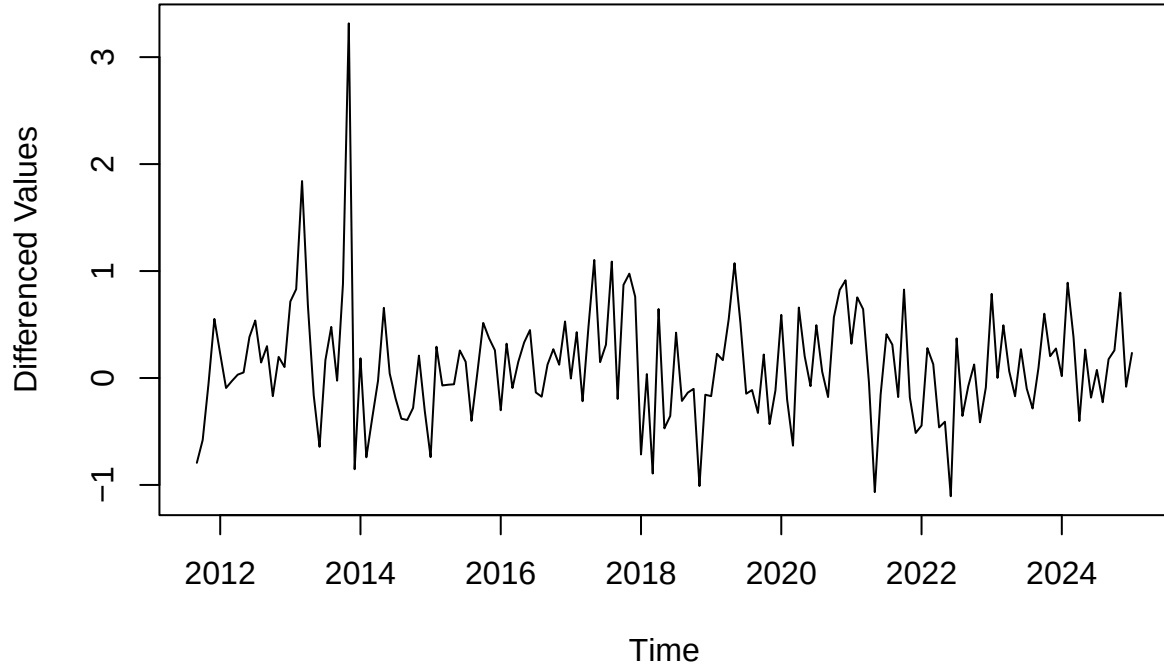
The results of Box-Cox transformation improved the distributional metrics of the series. The transformed series generated a Q-Q plot that fell better alongside the theoretical normal line while showing diminished data skewness in Figure 8. The altered dataset remained non-stationary with an ADF test p-value of 0.99 but attained better variance stability together with stronger conformity to normality as seen on visual tests and displayed by the Q-Q plot's reduced distorting factors.

The improvements are necessary because ARIMA requires stationary processes with constant variance and normal distributed residuals. The Box-Cox transformation stands as an important first step which leads to better model assumption fulfillment even though additional differencing is required for complete stationarity.

4.2 Order of Differencing

Since the series is non-stationary even after Box-Cox transformation, differencing will be required before ARIMA modeling.

Figure 9. First-Order Differenced Box-Cox Transformed Data

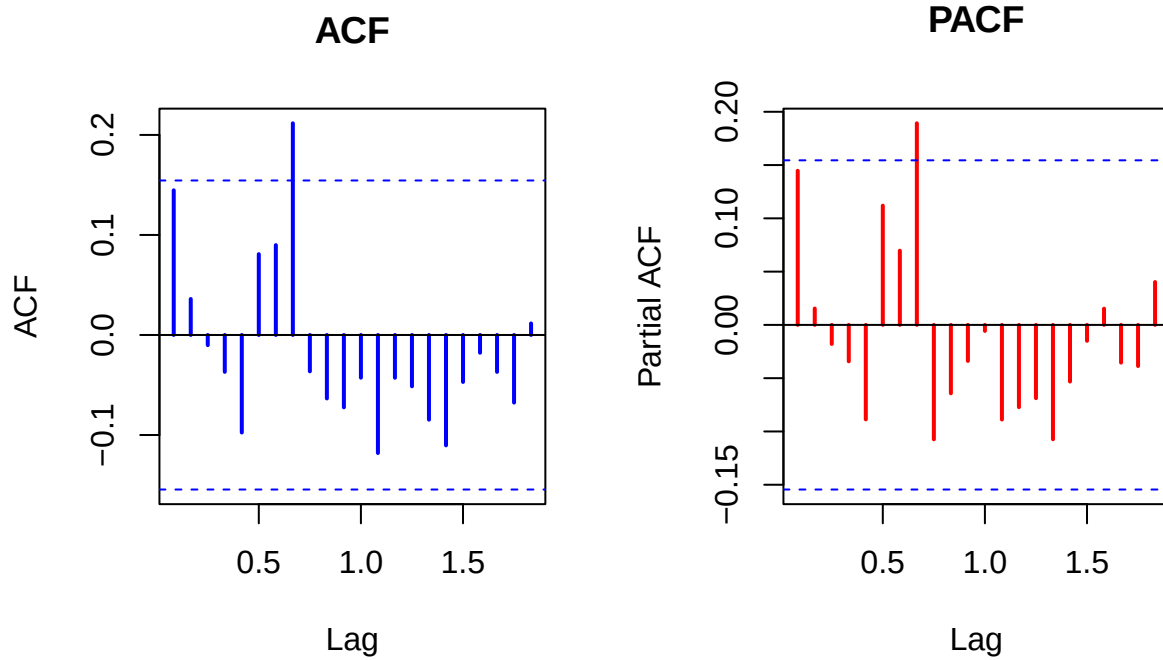


The Figure 9 illustrates a differenced version of the Box-Cox transformed Bitcoin Index time series data. Several important attributes emerge from visual examination of the information:

- **Trend:** The time series exhibits random movements instead of an upward or downward drift thus maintaining its constant mean.
- **Changing Variance:** The series displays stable variations throughout time therefore indicating homoscedasticity both after transformation and differencing steps.
- **Seasonality:** This time series series shows no patterns recurrently recurring throughout the year.
- **Autocorrelation:** The irregular pattern of fluctuations shows random behavior thus matching a stationary process idea.
- **Intervention:** The series shows no noticeable major external disturbances or structural changes when visually examined.

Multiple tests confirm that the first-order differenced Box-Cox transformed series displays stationary behavior. The time series pattern of differenced data excludes visual evidence of mean or variance instability over time indicating stationary behavior.

Figure 10 – ACF & PACF Plot of First Differenced Bitcoin Series



The ACF and PACF of the differenced Box-Cox transformed series are shown in Figure 10. The provided plots serve to strengthen the case for stationarity. The variance in the ACF suddenly decreases right after the first lag before remaining uniformly small throughout the rest of the sequence. The PACF demonstrates statistically important nodes at lag positions 1 and 2 while remaining under the significance area boundary. First difference and Box-Cox transform of the series display characteristics that match those of stationary processes.

ADF test on differenced series: Statistic = -4.896 , p-value = 0.01

The Augmented Dickey-Fuller (ADF) test results in a -4.896 test statistic while the p-value reaches 0.01. The test results enable us to reject the null hypothesis that a unit root exists at the 5% significance threshold which validates first difference stationarity of the analyzed series.

An ARIMA model becomes suitable after applying first-order differencing to the Box-Cox transformed Bitcoin index series. The implementation of proposed ARIMA(p, d, q) models requires d=1 since it represents the application of first-order differencing.

4.3 Model Identification using ACF and PACF

From Figure 10, both the ACF and PACF plot show similar structure. Both plots reveal a single significant autocorrelation together with a borderline significant in their patterns.

Multiple proposed models are available from the ACF and PACF analysis which include { ARIMA(1,1,1), ARIMA(2,1,2), ARIMA(2,1,1), ARIMA(1,1,2) }.

4.4 Extended ACF (EACF) Analysis

The EACF serves as a diagnostic model which determines correct AR and MA parameters from an AR-MA matrix by finding significant autocorrelations.

The EACF outputs for transformed and differenced Bitcoin Index data appear in Table 3. The table depicts possible combinations between AR terms and MA terms that reach up to lag 5.

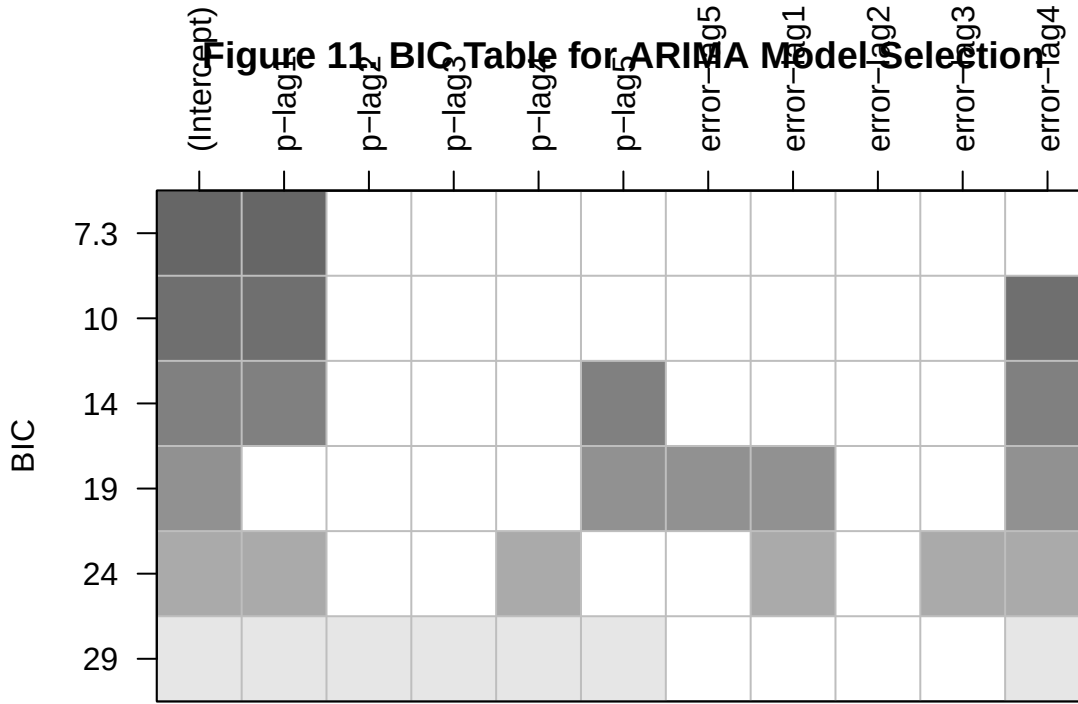
```
## AR/MA
##   0 1 2 3 4 5
## 0 o o o o o o
## 1 o o o o o o
## 2 x o o o o o
## 3 x o o o o o
## 4 x o x o o o
## 5 x x x o x o
```

A single top-left ‘o’ forms in the EACF table (Table 3) which starts at position (0,1). ARIMA(0,1,1) can be an appropriate model alongside different combinations of ARIMA(1,1,1), ARIMA(1,1,2), ARIMA(1,1,0), and ARIMA(2,1,0).

4.5 BIC Table

Model identification through the Bayesian Information Criterion penalizes model complexity while identifying the simplest model which effectively explains the data points.

```
## Reordering variables and trying again:
```



From the BIC table (**Figure 11**), we can capture the suggested model - ARIMA(1,1,0) and ARIMA(1,1,4)

The model identification utilizing Figure 10 ACF/PACF results alongside Table 3 EACF and Figure 11 BIC table produces Figure 11 (ARIMA(1,1,0) and ARIMA(1,1,4)) as potential models. It includes:

{ ARIMA(1,1,1), ARIMA(2,1,2), ARIMA(2,1,1), ARIMA(1,1,2), ARIMA(0,1,2), ARIMA(1,1,0), ARIMA(0,1,1), ARIMA(1,1,4)}

5 Fit selected models

We now fit the selected models using `arima()` and test the significance of parameters using `coeftest()`

5.1 Coefficient significance

The evaluation of candidate ARIMA models for statistical validity required an automated testing and estimation process built in R. The estimation of models using R's `arima()` function occurred through three different execution modes consisting of maximum likelihood (ML) and conditional sum of squares (CSS) and their combination (CSS-ML).

The `coeftest()` evaluated the significance of all parameters which we estimated utilizing ML together with CSS. A model was retained only if:

1. The statistical significance of the coefficients reached $p < 0.05$ under both ML and CSS methods, or
2. We recomputed the model parameters using the CSS-ML hybrid estimation approach unless all variables remained significant under this hypothesis before finally choosing the model.

Table 3: Parameter Estimates and Significance Tests for Final ARIMA Models

Model	Method	Parameter	Estimate	StdError	tValue	pValue
ARIMA(1,1,0)	ML & CSS	ar1	0.1885	0.0777	2.4262	0.0153
ARIMA(0,1,1)	ML & CSS	ma1	0.1673	0.0727	2.3019	0.0213
ARIMA(2,1,2)	CSS-ML	ar1	0.8923	0.0934	9.5552	0.0000
ARIMA(2,1,2)	CSS-ML	ar2	-0.8230	0.1339	-6.1482	0.0000
ARIMA(2,1,2)	CSS-ML	ma1	-0.7934	0.0948	-8.3706	0.0000
ARIMA(2,1,2)	CSS-ML	ma2	0.8758	0.1269	6.9005	0.0000

The results are summarized below:

- **ARIMA(1,1,0)**: Estimated via ML and CSS. The statistical analysis showed that the AR(1) coefficient proved significant at the 0.0153 level thus indicating moderate yet meaningful autoregressive short-term patterns. The retention of the selected model became possible because both estimation methods produced matching results.
- **ARIMA(0,1,1)**: The MA(1) coefficient proved significant at $p = 0.0213$ when evaluated by both ML and CSS approaches thus providing confirmation for retention of this model structure. This model shows evidence of short-term memory functions because it uses lagged forecast errors to create its effect.
- **ARIMA(2,1,2)**: ARIMA(2,1,2) showed conflicting significance results when using ML and CSS which led to estimation with the CSS-ML procedure to reach a final model specification. The test results under this method demonstrated statistical significance of all coefficients including AR1, AR2, MA1, MA2 ($p < 0.001$) thus confirming this model as an effective temporal predictor with its dual autoregressive and moving average structure.

The finding verifies that the three models ARIMA(1,1,0), ARIMA(0,1,1) and ARIMA(2,1,2) are sufficient.

5.2 Model Selection Based on Information Criteria

The Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC) calculations gave us an objective way to evaluate and select from the retained ARIMA models based on parsimony and effectiveness. Smaller values of AIC and BIC demonstrate superior model performance in terms of fit quality together with penalty for model complexity.

We conducted parameter re-estimation of the three accepted models—ARIMA(1,1,0), ARIMA(0,1,1), and ARIMA(2,1,2)—through their optimal fitting approaches (ML for first two, CSS-ML for last) before computing their information criterion values.

Table 4: ARIMA Models Sorted by AIC

Model	AIC	BIC
ARIMA(1,1,0)	260.37	266.53
ARIMA(0,1,1)	261.05	267.21
ARIMA(2,1,2)	265.87	281.27

Table 4 demonstrates that the ARIMA(1,1,0) model achieved the lowest AIC and BIC values which indicates its selection as the most suitable model according to both criteria. ARIMA(0,1,1) provided similar

information criteria to its competitor model yet added a different model structure (moving average instead of autoregressive) without any significant benefits. The ARIMA(2,1,2) model received higher AIC and BIC values because it had additional parameters although it was statistically valid and flexible.

The analysis retained three ARIMA models including ARIMA(1,1,0), ARIMA(0,1,1) and ARIMA(2,1,2) for subsequent evaluation step. The statistical significance tests of all coefficients proved positive after re-estimation with the optimal fitting methods ML or CSS-ML.

5.3 Forecast Accuracy Evaluation

The out-of-sample forecast accuracy was evaluated for the Bitcoin Index using the three candidate models—ARIMA(1,1,0), ARIMA(0,1,1) and ARIMA(2,1,2)—via `accuracy()` from `forecast` in R.

Table 5: Forecast Accuracy Metrics for ARIMA Models

	Model	RMSE	MAE	MAPE	MASE
3	ARIMA(2,1,2)	0.5339	0.3873	1.8330	0.1756
1	ARIMA(1,1,0)	0.5347	0.3878	1.8315	0.1758
2	ARIMA(0,1,1)	0.5359	0.3868	1.8277	0.1754

Statistical data about the three candidate model forecast accuracies can be found in Table 5. The performance results obtained from these candidate models exhibit similar outcomes for each metric of error evaluation.

- **RMSE (Root Mean Square Error):** The lowest RMSE value appears in ARIMA(0,1,1) although ARIMA(1,1,0) displays a minimal difference in RMSE measurement.
- **MAE (Mean Absolute Error):** The three models possess nearly identical values of Mean Absolute Error.
- **MAPE (Mean Absolute Percentage Error):** The models exhibit highly comparable percentage error values according to MAPE (Mean Absolute Percentage Error) measurements.
- **MASE (Mean Absolute Scaled Error):** The ARIMA(0,1,1) model demonstrates minor superiority on this scale-independent metric.

Research shows that ARIMA(0,1,1) produced slightly superior accuracy metrics yet the differences overlap within statistical significance levels. The Bitcoin Index series uses ARIMA(1,1,0) as its preferred model because it has both the optimal AIC and BIC values shown in Tables 4 and 5 and requires only a small number of parameters.

5.4 Summary of Model Fitting

The best model choice became ARIMA(1,1,0) because it possesses these advantages over ARIMA(0,1,1) and ARIMA(2,1,2):

- Statistically significant coefficients under ML and CSS
- Lowest AIC and BIC values
- Competitive and nearly identical forecast accuracy
- Simplicity and parsimony

The model provides both statistical strength and prediction accuracy and interpretability that makes it the optimal choice for modeling Bitcoin Index data.

6 Conclusion

Time series analysis was used to examine the monthly Bitcoin Index spanning from August 2011 to January 2025 for selecting an appropriate ARIMA model to model and forecast future prices.

6.1 Key Findings

1. **Time Series Characteristics:** The Bitcoin Index demonstrates two major time series properties: strong non-stationary behavior accompanied by an upward trend together with heteroscedasticity characteristics. Stationarity required both transformation through Box-Cox as well as differencing to proceed with modeling.
2. **Optimal Model:** Systematic model assessment resulted in ARIMA(1,1,0) becoming the selected model among all tested candidates. The selected model provided the optimal combination between information criteria fit measurements and minimal parameter count. After first differencing the transformed data the Bitcoin Index operates based on a first-order autoregressive process according to the model.

6.2 Limitations

Several limitations should be acknowledged:

1. **Exogenous Factors:** The analysis employing ARIMA solely assesses time-based dynamics from the Bitcoin Index data neglecting outside elements like regulatory and market conditions and technological advancements.
2. **Structural Breaks:** The ARIMA model does not include built-in functionality to detect structural changes detected within the Bitcoin time series data although such occurrences emerged from the analysis. The analysis could benefit from advanced models such as regime-switching models to handle these particular dynamics.

6.3 Future Research Directions

Building on this analysis, several avenues for future research emerge:

1. **ARIMAX Models:** Adding external variables such as stock market indices and economic indicators and gold prices to ARIMAX Models will improve the predictive capability of the model.
2. **Volatility Modeling:** The apparent heteroscedasticity condition makes it possible to link GARCH models with ARIMA structure for modeling both mean and variance patterns.
3. **Non-Linear Models:** Non-linear time-series models should be examined since they could provide superior modelling of cryptocurrency market behaviors.
4. **Multivariate Analysis:** The investigation can incorporate multiple cryptocurrency data to detect shared market forces which shape the entire crypto market.

The time series analysis delivers significant information about Bitcoin Index temporal characteristics while developing a statistical approach to predict historical and future trends. The ARIMA(1,1,0) model represents an effective simple framework to understand the major patterns of the series which provides a strong basis for building more sophisticated approaches moving forward.

7 Appendix: R Code

```
# Setup and library loading
library(tseries)
library(forecast)
library(ggplot2)
library(dplyr)
library(knitr)
library(kableExtra)
library(stats)
library(psych)
library(zoo)
library(lmtest)
library(TSA)

# Creating a reusable function for ACF and PACF plotting to avoid code repetition
plot_acf_pacf <- function(data, fig_num = NULL, title = "ACF and PACF Plots") {
  par(mfrow = c(1, 2))
  par(oma = c(0, 0, 3, 0))

  if (!is.null(fig_num)) {
    main_title <- paste("Figure", fig_num, "-", title)
  } else {
    main_title <- title
  }

  acf(data, main = "ACF", col = "blue", lwd = 2)

  pacf(data, main = "PACF", col = "red", lwd = 2)

  mtext(main_title, side = 3, line = 1, outer = TRUE, cex = 1.2, font = 2)

  par(mfrow = c(1, 1))
}

# Read and check the data
data <- read.csv("assignment2Data2025.csv")
head(data)
class(data)

# Create time series object
# The data starts from August 2011
data.ts <- ts(data$Bitcoin, start = c(2011, 8), frequency = 12)

# Data Summary
str(data.ts)
sum(is.na(data.ts))

# Computing descriptive statistics for the Bitcoin time series
summary_stats <- describe(data.ts)
summary_stats_subset <- summary_stats[, c("mean", "sd", "min", "max", "skew", "kurtosis")]

summary_stats_subset %>%
```

```

kable(digits = 2, caption = "Table 1. Descriptive Statistics of Bitcoin Index") %>%
kable_styling(bootstrap_options = c("striped", "hover", "condensed"), full_width = FALSE)

# Creating a time series plot
plot(data.ts, ylab='Bitcoin Index (USD)',
      main='Figure 1. Time Series Plot of the Bitcoin Index',
      col = "darkblue", lwd = 1.5)

# Decomposing the time series into trend, seasonal, and remainder components
decomp <- stl(data.ts, s.window = "periodic")
plot(decomp, main="Figure 2. STL Decomposition of Bitcoin Index")

# Creating a scatter plot of neighboring pairs
plot(
  y = data.ts,
  x = zlag(data.ts),
  xlab = "Lag 1 (Previous Month Bitcoin Index)",
  ylab = "Current Month Bitcoin Index",
  main = "Figure 3. Scatter Plot of Neighboring Pairs (Lag 1)",
  pch = 19,
  col = "darkblue",
  cex = 0.7
)
abline(lm(data.ts ~ zlag(data.ts)), col = "red", lwd = 2)

# Correlation
y = data.ts
x = zlag(data.ts)
index = 2:length(x)
cor_value <- cor(y[index], x[index])
cat("Correlation between current and previous month Bitcoin values:", round(cor_value, 4))

# Computing rolling statistics to assess stationarity
rolling_mean <- rollmean(data.ts, k = 12, fill = NA)
rolling_sd <- rollapply(data.ts, width = 12, FUN = sd, fill = NA)

# Creating a plot with rolling statistics
plot(data.ts, type = "l", main="Figure 4. Rolling Statistics of Bitcoin Index",
      ylab = "Bitcoin Index (USD)")
lines(rolling_mean, col = "red")
lines(rolling_sd, col = "blue")

legend("topleft", legend = c("Original Data", "Rolling Mean", "Rolling SD"),
      col = c("black", "red", "blue"), lty = 1, cex = 0.8, bty = "n")

# Conducting formal stationarity tests
adf_test <- adf.test(data.ts)
kpss_test <- kpss.test(data.ts)
pp_test <- pp.test(data.ts)

# Creating a table of test results
test_results <- data.frame(
  Test = c("Augmented Dickey-Fuller", "KPSS", "Phillips-Perron"),

```



```

Statistic = c(adf_test$statistic, kpss_test$statistic, pp_test$statistic),
p_value = c(adf_test$p.value, kpss_test$p.value, pp_test$p.value),
Null_Hypothesis = c("Non-stationary", "Stationary", "Non-stationary"),
Conclusion = c("Fail to Reject", "Reject", "Fail to Reject")
)

kable(test_results, caption = "Table 2. Results of Stationarity Tests") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"), full_width = FALSE)

# Using the custom function to create ACF and PACF plots
plot_acf_pacf(data.ts, fig_num = 5, title = "Bitcoin Index ACF & PACF")

# Creating a Q-Q plot and conducting Shapiro-Wilk test
qqnorm(data.ts, main = "Figure 6. Normal Q-Q Plot of the Original Time Series")
qqline(data.ts, col = 2)
shapiro_test <- shapiro.test(data.ts)
cat("Shapiro-Wilk normality test: W =", round(shapiro_test$statistic, 4),
    ", p-value =", format.pval(shapiro_test$p.value, digits = 4))

# Checking for minimum value to ensure all data is positive
min(data.ts)

# Box-Cox transformation
BC <- BoxCox.ar(y = data.ts, lambda = seq(0, 0.5, 0.01))

# Extracting the optimal lambda value and confidence interval
BC$ci
lambda <- BC$lambda[which(max(BC$loglike) == BC$loglike)]
lambda

# Applying the Box-Cox transformation with the optimal lambda
datatsBC <- (data.ts^lambda - 1) / lambda

# Plotting the transformed series
plot(datatsBC, type = "l", col = "blue",
     main = "Figure 7. Box-Cox Transformed Time Series",
     ylab = "Transformed Values", xlab = "Time")

# Creating a Q-Q plot for the transformed series
qqnorm(datatsBC, main = "Figure 8. Normal Q-Q Plot for the Box-Cox transformed series")
qqline(datatsBC, col = 2)

# Testing normality and stationarity of the transformed series
shapiro_test_bc <- shapiro.test(datatsBC)
cat("Shapiro-Wilk normality test after Box-Cox: W =", round(shapiro_test_bc$statistic, 4),
    ", p-value =", format.pval(shapiro_test_bc$p.value, digits = 4))

adf_test_bc <- adf.test(datatsBC)
cat("\nADF test after Box-Cox: Statistic =", round(adf_test_bc$statistic, 4),
    ", p-value =", format.pval(adf_test_bc$p.value, digits = 4))

# Applying first-order differencing to the transformed series
diff_boxcox_datats <- diff(datatsBC, differences = 1)

```

```

# Plotting the differenced series
plot(diff_boxcox_datats, type = "l",
     main = "Figure 9. First-Order Differenced Box-Cox Transformed Data",
     xlab = "Time", ylab = "Differenced Values")

# Creating ACF and PACF plots for the differenced series
plot_acf_pacf(diff_boxcox_datats, fig_num = 10, title = "ACF & PACF Plot of the First Differenced Bitcoi

# Conducting ADF test on the differenced series
adf_diff <- adf.test(diff_boxcox_datats)
cat("ADF test on differenced series: Statistic =", round(adf_diff$statistic, 4),
    ", p-value =", format.pval(adf_diff$p.value, digits = 4))

# Creating a function for EACF display and analysis
eacf_display <- function(x, ar.max = 5, ma.max = 5) {
  eacf_result <- TSA::eacf(x, ar.max = ar.max, ma.max = ma.max)
  return(eacf_result)
}

# Applying EACF to the differenced series
eacf_diff <- eacf_display(diff_boxcox_datats)

# Creating BIC table using armasubsets
res = armasubsets(y=diff_boxcox_datats, nar = 5, nma = 5, y.name = 'p', ar.method = 'ols')
plot(res, main = "Figure 11. BIC Table for ARIMA Model Selection")

## Define candidate models
candidate_models <- list(
  "ARIMA(0,1,2)" = c(0, 1, 2),
  "ARIMA(1,1,0)" = c(1, 1, 0),
  "ARIMA(0,1,1)" = c(0, 1, 1),
  "ARIMA(1,1,1)" = c(1, 1, 1),
  "ARIMA(1,1,2)" = c(1, 1, 2),
  "ARIMA(2,1,1)" = c(2, 1, 1),
  "ARIMA(2,1,2)" = c(2, 1, 2),
  "ARIMA(1,1,4)" = c(1, 1, 4)
)

## Function to check coefficient significance
all_significant <- function(coeftest_result) {
  all(coeftest_result[, 4] < 0.05)
}

## Evaluate models
final_models <- list()

for (model_name in names(candidate_models)) {
  order <- candidate_models[[model_name]]

  # Try ML
  fit_ml <- tryCatch(arima(datatsBC, order = order, method = "ML"), error = function(e) NULL)
  sig_ml <- !is.null(fit_ml) && all_significant(coeftest(fit_ml))

```

```

# Try CSS
fit_css <- tryCatch(arima(datatsBC, order = order, method = "CSS"), error = function(e) NULL)
sig_css <- !is.null(fit_css) && all_significant(coeftest(fit_css))

if (sig_ml && sig_css) {
  final_models[[model_name]] <- list(model = fit_ml, method = "ML & CSS")
} else if (!sig_ml && !sig_css) {
  next # Drop model
} else {
  # Try CSS-ML if there's disagreement
  fit_cssml <- tryCatch(arima(datatsBC, order = order, method = "CSS-ML"), error = function(e) NULL)
  if (!is.null(fit_cssml) && all_significant(coeftest(fit_cssml))) {
    final_models[[model_name]] <- list(model = fit_cssml, method = "CSS-ML")
  }
}
}

## Create a table to display coefficient test results for final models
model_results <- data.frame(
  Model = character(),
  Method = character(),
  Parameter = character(),
  Estimate = numeric(),
  StdError = numeric(),
  tValue = numeric(),
  pValue = numeric(),
  stringsAsFactors = FALSE
)

for (name in names(final_models)) {
  model_obj <- final_models[[name]]$model
  ct <- coeftest(model_obj)
  method <- final_models[[name]]$method

  for (i in 1:nrow(ct)) {
    param_name <- rownames(ct)[i]
    model_results <- rbind(model_results, data.frame(
      Model = name,
      Method = method,
      Parameter = param_name,
      Estimate = ct[i, 1],
      StdError = ct[i, 2],
      tValue = ct[i, 3],
      pValue = round(ct[i, 4], digits = 4),
      stringsAsFactors = FALSE
    ))
  }
}

kable(model_results, digits = 4,
      caption = "Table 4. Parameter Estimates and Significance Tests for Final ARIMA Models") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"), full_width = FALSE)

```

```

# Refit retained models (use appropriate estimation method as previously determined)
fit_110 <- arima(datatsBC, order = c(1,1,0), method = "ML")
fit_011 <- arima(datatsBC, order = c(0,1,1), method = "ML")
fit_212 <- arima(datatsBC, order = c(2,1,2), method = "ML")

# Create comparison table
model_info <- data.frame(
  Model = c("ARIMA(1,1,0)", "ARIMA(0,1,1)", "ARIMA(2,1,2)"),
  AIC = c(AIC(fit_110), AIC(fit_011), AIC(fit_212)),
  BIC = c(BIC(fit_110), BIC(fit_011), BIC(fit_212))
)

# Sort by AIC and BIC (separately)
model_info_aic_sorted <- model_info[order(model_info$AIC), ]

# Display tables
kable(model_info_aic_sorted, digits = 2,
      caption = "Table 5. ARIMA Models Sorted by AIC") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"), full_width = FALSE)

# Fit models using forecast::Arima with appropriate methods
fit_f_110 <- Arima(datatsBC, order = c(1,1,0), method = "ML")
fit_f_011 <- Arima(datatsBC, order = c(0,1,1), method = "ML")
fit_f_212 <- Arima(datatsBC, order = c(2,1,2), method = "ML")

# Compute accuracy metrics
acc_110 <- accuracy(fit_f_110)
acc_011 <- accuracy(fit_f_011)
acc_212 <- accuracy(fit_f_212)

# Build summary table
forecast_accuracy <- data.frame(
  Model = c("ARIMA(1,1,0)", "ARIMA(0,1,1)", "ARIMA(2,1,2)"),
  RMSE = c(acc_110["Training set", "RMSE"], acc_011["Training set", "RMSE"], acc_212["Training set", "RMSE"]),
  MAE = c(acc_110["Training set", "MAE"], acc_011["Training set", "MAE"], acc_212["Training set", "MAE"]),
  MAPE = c(acc_110["Training set", "MAPE"], acc_011["Training set", "MAPE"], acc_212["Training set", "MAPE"]),
  MASE = c(acc_110["Training set", "MASE"], acc_011["Training set", "MASE"], acc_212["Training set", "MASE"])
)

# Sort by RMSE
forecast_accuracy_sorted <- forecast_accuracy[order(forecast_accuracy$RMSE), ]

# Display
kable(forecast_accuracy_sorted, digits = 4,
      caption = "Table 6. Forecast Accuracy Metrics for ARIMA Models") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"), full_width = FALSE)

```